Level 1 - Syntax, Semantics, and Program Correctness

Jaymin C. Amin

School of Science and Technology, United States International University – Africa

APT1030A: Fundamentals of Programming Languages

Dr. Lawrence Nderu

March 22, 2026

Level 1 - Syntax, Semantics, and Program Correctness

Context: eCitizen Login Validation System

Concepts

- Syntax vs semantics
- Compilation vs interpretation
- Runtime errors

Implementation Task

Write a login validation program in:

- Python
- Java
- C++

The program should:

1. Ask for username and password.
2. Validate against predefined credentials.
3. Print:
    - "Access Granted"
    - "Invalid Credentials"

Example Requirement:

In all languages:

- *Hardcode:*
- *username = "adminKE"*
- *password = "254Secure"*

Analytical Question

1. Introduce a syntax error in each language and document:

    - Error message
    - Type of error

Python

In Python, a missing colon at the end of an if statement is a syntax error.

- **The Change:** if user_input == VALID_USER (Missing the :)
- **Error Message:** SyntaxError: expected ':'
- **Type of Error:** Interpreted Syntax Error

Java

Java requires a semicolon to terminate statements.

↓ Next Page ↓

- **The Change:** String validUser = "adminKE" (Removing the ;)
- **Error Message:** error: ';' expected
- **Type of Error:** Compilation Error.

C++

In C++, using a variable that hasn't been declared is a syntax/semantic violation.

- **The Change:** std::cin >> userinput; (Changed the 'I' to lowercase 'i' in userInput).
- **Error Message:** error: 'userinput' was not declared in this scope
- **Type of Error:** Compilation Error.

2. Introduce a logical error (wrong comparison operator) and explain:

    - Why the program compiles but behaves incorrectly.

A **Logical Error** happens when the syntax is perfect, but the logic is flawed. The program runs, but it doesn't do what you want.

For example, changing the comparison operator from == (equals) to != (does not equal):

```
if user_input != VALID_USER or pass_input != VALID_PASS:
    print("Access Granted")
```

To the compiler or interpreter, != is a perfectly valid operator. It doesn't know you *intended* to check for equality; it just sees a request to check for inequality. This is a failure of **Semantics**. The "meaning" of your code has shifted.